

Genetic Algorithm (GA) – Lecture Notes

Artificial Intelligence Basics

1. Introduction to Genetic Algorithms

A **Genetic Algorithm (GA)** is a search and optimization technique inspired by the process of **natural evolution**.

It is part of **Artificial Intelligence (AI)** and **Evolutionary Computing**.

Key Idea:

“Survival of the fittest” – better solutions are more likely to survive and improve over time.

2. Biological Inspiration

Genetic Algorithms mimic biological concepts:

Biological Concept	GA Equivalent
Chromosome	Solution
Gene	Part of solution
Population	Set of solutions
Fitness	Quality of solution
Natural Selection	Selection process
Crossover	Recombination
Mutation	Random variation

3. Basic Terminology

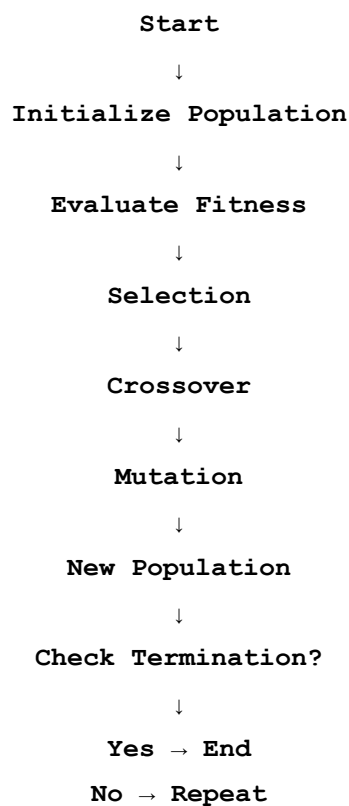
- **Population:** A group of candidate solutions.
- **Chromosome:** Representation of a solution (often a string or array).
- **Gene:** A single value in a chromosome.
- **Fitness Function:** Measures how good a solution is.
- **Generation:** One iteration of the algorithm.

4. Working of Genetic Algorithm

Step-by-Step Process:

1. **Initialize Population**
 - Generate random solutions.
2. **Evaluate Fitness**
 - Use a fitness function to score each solution.
3. **Selection**
 - Select the best solutions for reproduction.
4. **Crossover (Recombination)**
 - Combine two parents to create offspring.
5. **Mutation**
 - Randomly change some genes to maintain diversity.
6. **Replacement**
 - Form a new population.
7. **Termination**
 - Stop when:
 - Maximum generations reached, OR
 - Optimal solution found.

5. Flow Diagram (Conceptual)



6. Selection Methods

- **Roulette Wheel Selection**
- **Tournament Selection**
- **Rank Selection**

7. Crossover Techniques

- **Single-point crossover**
- **Two-point crossover**
- **Uniform crossover**

Example:

```
Parent 1: 1100  
Parent 2: 0011
```

```
After crossover:  
Child 1: 1101  
Child 2: 0010
```

8. Mutation

- Small random changes in genes.
- Prevents algorithm from getting stuck in local optimum.

Example:

```
Before: 1100  
After : 1101
```

9. Fitness Function

- Defines how good a solution is.
- Problem-specific.

Example:

- Maximizing profit
- Minimizing error
- Shortest path

10. Advantages of Genetic Algorithms

- Works well for complex problems
- Does not require gradient information
- Can escape local optima

- Flexible and adaptable

11. Disadvantages

- Computationally expensive
- May converge slowly
- Requires careful parameter tuning

12. Applications in AI

- Optimization problems
- Machine learning model tuning
- Scheduling problems
- Robotics
- Game AI
- Feature selection

13. Simple Example

Problem:

Maximize function:

$$f(x) = x^2$$

Steps:

- Encode x in binary
- Generate random values
- Apply GA operations
- Find best x value over generations

14. Key Parameters

- Population size
- Mutation rate
- Crossover rate
- Number of generations

15. Summary

- Genetic Algorithms are inspired by evolution.
- They improve solutions over generations.
- Useful for optimization and search problems.

- Widely used in Artificial Intelligence.

16. Short Questions (For Students)

1. What is a genetic algorithm?
2. Define fitness function.
3. What is crossover?
4. Why is mutation important?
5. List applications of GA.

17. Assignment

- Implement a simple Genetic Algorithm in any programming language.
- Solve a basic optimization problem.

Examples:

- Simple Genetic Algorithm (SGA)
- Adaptive Genetic Algorithm (AGA)
- Steady-State Genetic Algorithm (SSGA)
- Micro Genetic Algorithm (μ GA)
- Parallel Genetic Algorithm (PGA)
- Hierarchical Genetic Algorithm (HGA)
- Real-Coded Genetic Algorithm (RCGA)
- Distributed Genetic Algorithm (DGA)
- Elitist Genetic Algorithm
- Multi-Objective Genetic Algorithm (MOGA)
- Island Genetic Algorithm
- Cultural Genetic Algorithm (CGA)
- Immune Genetic Algorithm (IGA)
- Hybrid Genetic Algorithm (HGA)
- Chaotic Genetic Algorithm (CGA)
- Genetic Programming (GP)
- Multi-Population Genetic Algorithm
- Age-Layered Population Structure (ALPS) GA
- Memetic Algorithm (MA)
- Differential Evolution Genetic Algorithm (DEGA)

GA (Genetic Algorithm) – आसान समझ

असली समस्या (संक्षेप में)

हम एक साधारण समस्या हल कर रहे हैं:

हमें एक संख्या x (0 से 31 के बीच) ढूंढनी है, जिससे $f(x) = x^2$ सबसे ज्यादा हो।

सरल शब्दों में

अगर:

$x = 5 \rightarrow f(x) = 25$
 $x = 10 \rightarrow f(x) = 100$
 $x = 20 \rightarrow f(x) = 400$
 $x = 31 \rightarrow f(x) = 961$ (सर्वोच्च)

तो सही उत्तर **आदर्श रूप से = 31**

प्रश्न: “कौन सा x लेने पर x^2 सबसे बड़ा मिलेगा?”

सामान्य तरीका बनाम GA (Genetic Algorithm)

सामान्य तरीका:

सबके सब मान चेक करो:

$0^2, 1^2, 2^2, \dots, 31^2$

छोटे उदाहरण में ठीक है, लेकिन बड़े मामलों में बहुत समय लगेगा।

Genetic Algorithm तरीका:

- शुरू में **random guesses**
 - सबसे अच्छे चुनें
 - उन्हें **मिलाएं (crossover)**
 - **थोड़ी random बदल** (mutation) डालें
 - धीरे-धीरे **बेहतर समाधान पाएँ**
-

GA समाधान कैसे काम कर रहा है

Random guesses से शुरुआत
उदाहरण:

[3, 10, 7, 25, 15, 2]

ये सभी संभावित उत्तर (x values) हैं।

देखो कौन सबसे अच्छा है (Fitness check)

$$3^2 = 9$$

$$10^2 = 100$$

$$25^2 = 625 \quad \checkmark \text{ सबसे अच्छा}$$

बड़े x → बेहतर समाधान

सबसे अच्छे चुनना (Selection)

[25, 15, 10]

कमजोर मान हटा दिए गए।

मिलाना (Crossover)

$$\text{Parent1} = 25, \text{Parent2} = 15$$

$$\text{Child} = (25 + 15) / 2 = 20$$

नए और बेहतर guesses बन रहे हैं।

थोड़ी random बदल (Mutation)

$$20 \rightarrow 22$$

नए संभावित समाधान खोजने के लिए।

दोहराना (Repeat for generations)

हर generation में:

Better → और Better → लगभग Best

अंतिम परिणाम

Best solution $\approx$ 29 (या 30 या 31)

GA ने लगभग सही उत्तर ढूंढ लिया।

आसान समझ

समस्या:

“सबसे बड़ा x^2 चाहिए।”

GA का काम:

“Random guesses को धीरे-धीरे सुधार कर सबसे अच्छा x ढूँढना।”

एक पंक्ति में

“ हम एक ऐसा नंबर खोज रहे हैं जिसका square सबसे बड़ा हो, और GA इसे समझदारी से खोजता है। ”